

Post-Partitioning Analysis and Debug

- The second exploration stage examining the partitioning effects
- Pre-partitioning architectural models are refined
- Hardware and software models executed together
- The models collectively should support the desired kind of analysis and debug
 - Too much detail - the analysis too slow or too late
 - Not enough detail - not provide the necessary information
- Optimization iteration back to the partitioning stage - design space exploration

Post-Partitioning Analysis Aspects

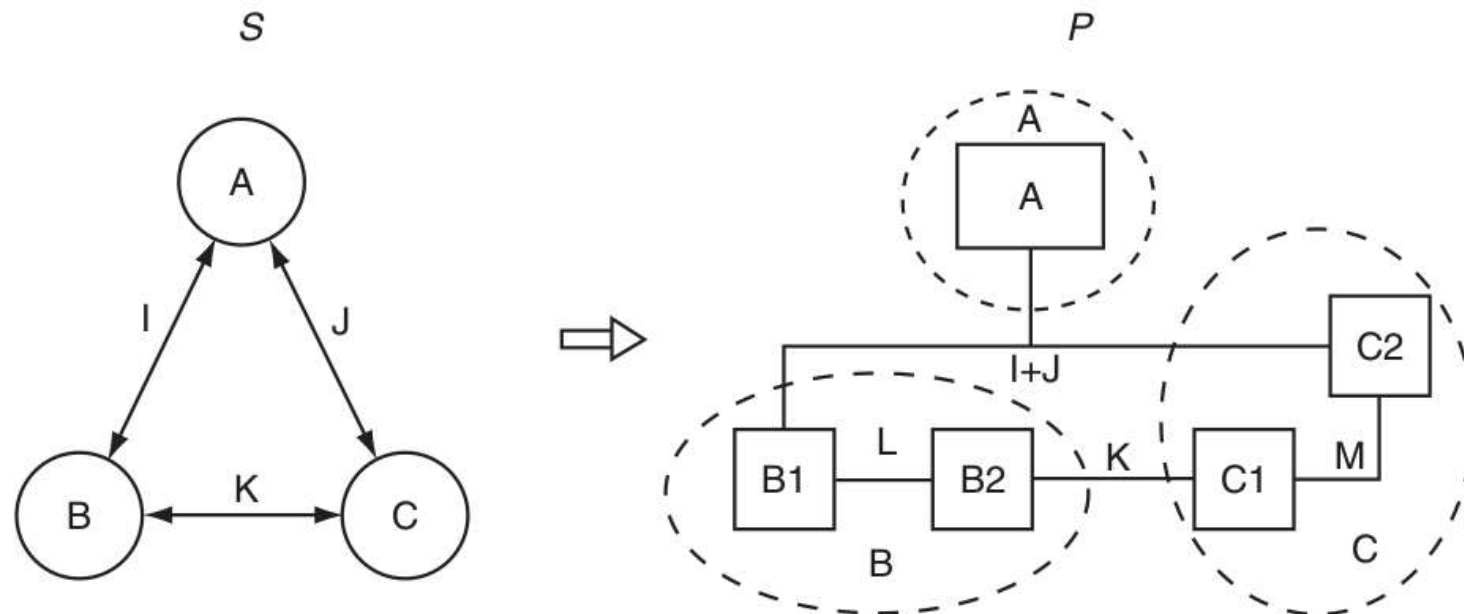
- Hardware and software modeling and co-modeling
- Performance, power, and cost estimates
- Managing abstraction of various models
- Dynamic and static analysis

Roles and Responsibilities

- Careful analysis and modeling done early
 - Implementation will merely eat into design margin
- The original design assumptions may prove to be false or inaccurate
 - E.G. Not possible to implement in hardware
 - E.G. Processor exceed predicted cycle count
 - Iterate back
 - Abandon design

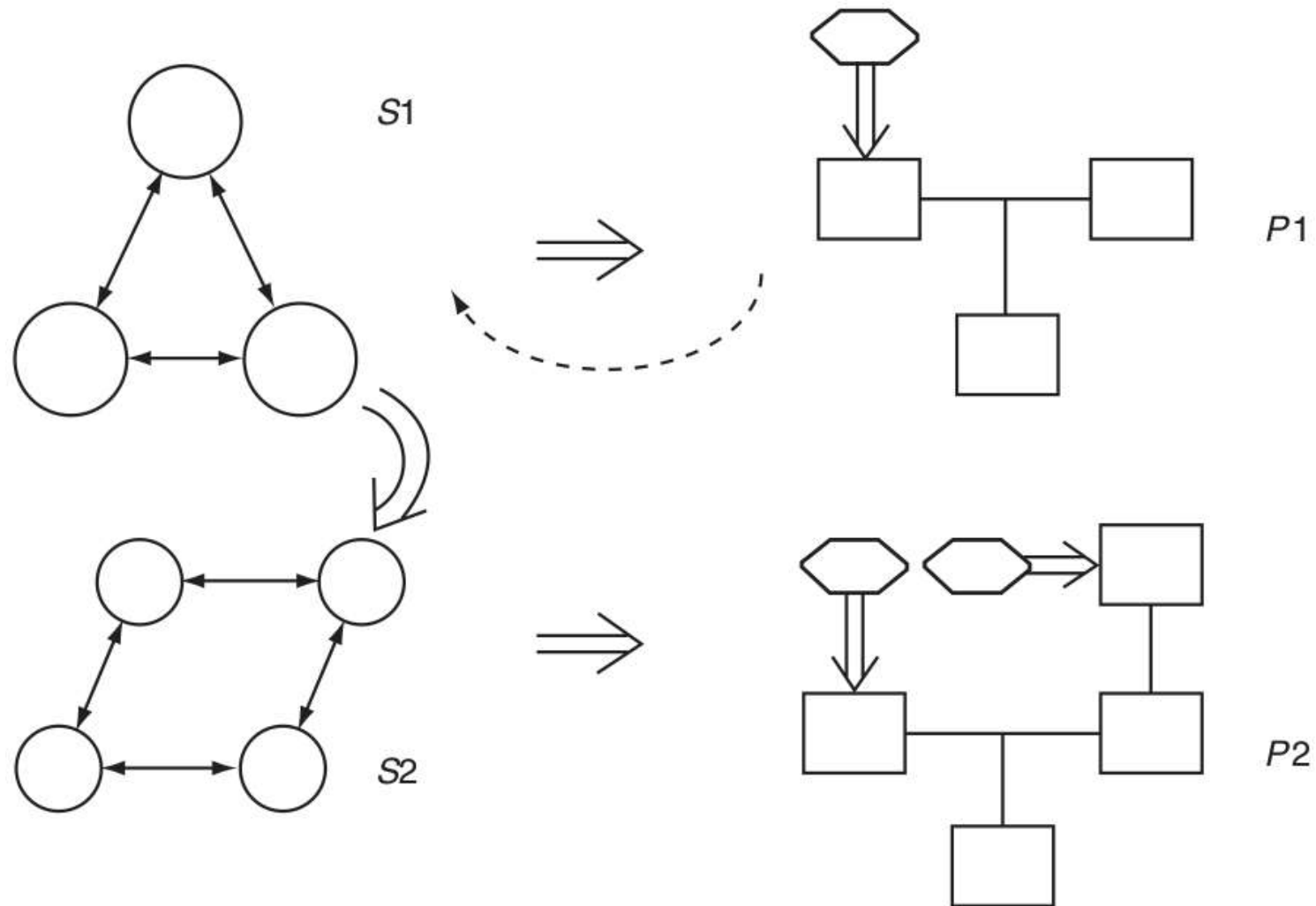
Interface Responsibilities

- Designer or instantiator



Model Maintenance

- Re-partitioning or re-specification -> maintenance order

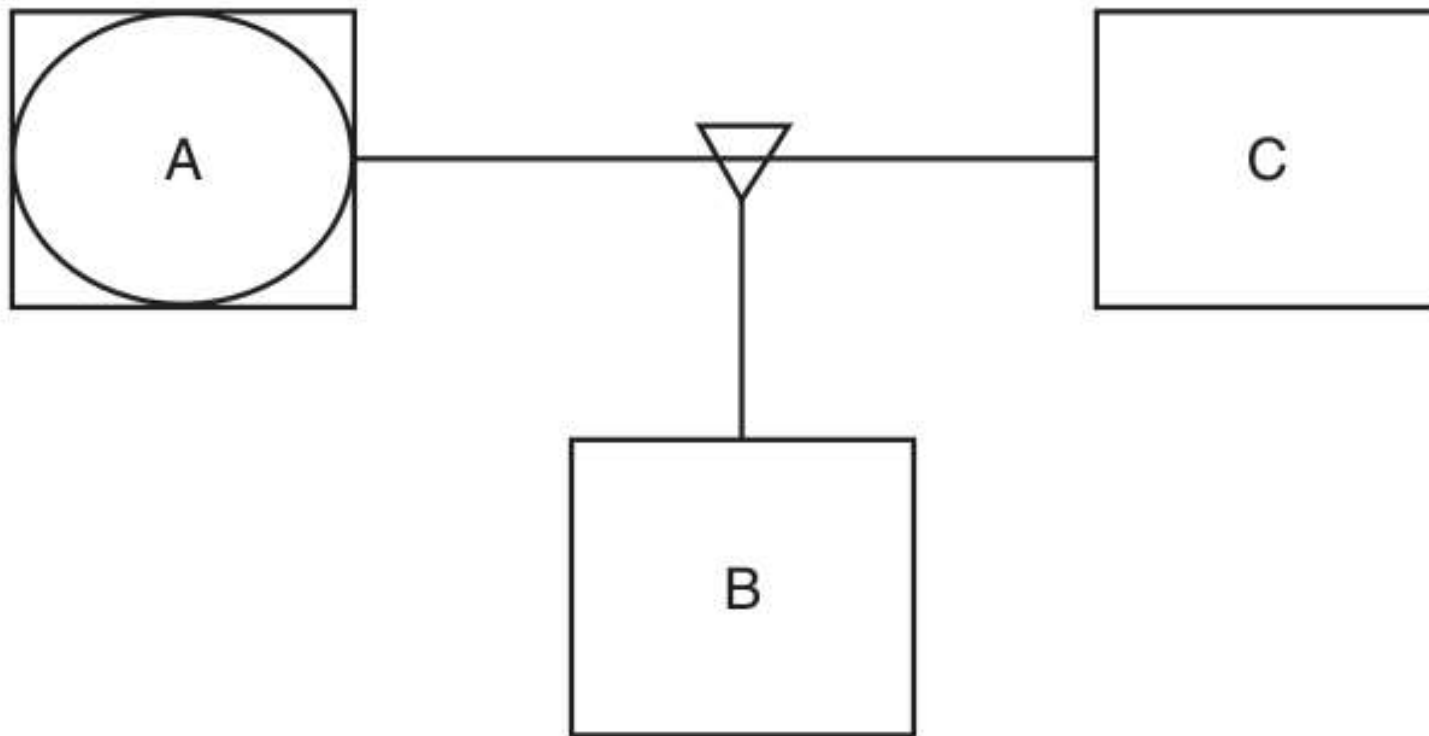


Hardware and Software Modeling and Co-Modeling

- Hardware and software execute cooperatively
- Need to communicate in a meaningful way
- Interfaces must be compatible
- Modeling options for the software components include the following:
 - Single model - Software together with the hardware
 - Filter and translate - Another component to filter software transactions and translate the communication
 - Separata Host Model - Model the software running in combination with the hardware on which it would execute.

Hardware and Software Modeling - Single Model

- The model for A is written such that it represents both the software and the hardware.



Hardware and Software Modeling - Single Model - Pros - Cons

- The model need not be defined in fine detail
- Extremely fast
 - Basis for virtual platforms
- The confusion
 - The melding (rather than separation) of concerns

Hardware and Software Modeling - Single Model - Management

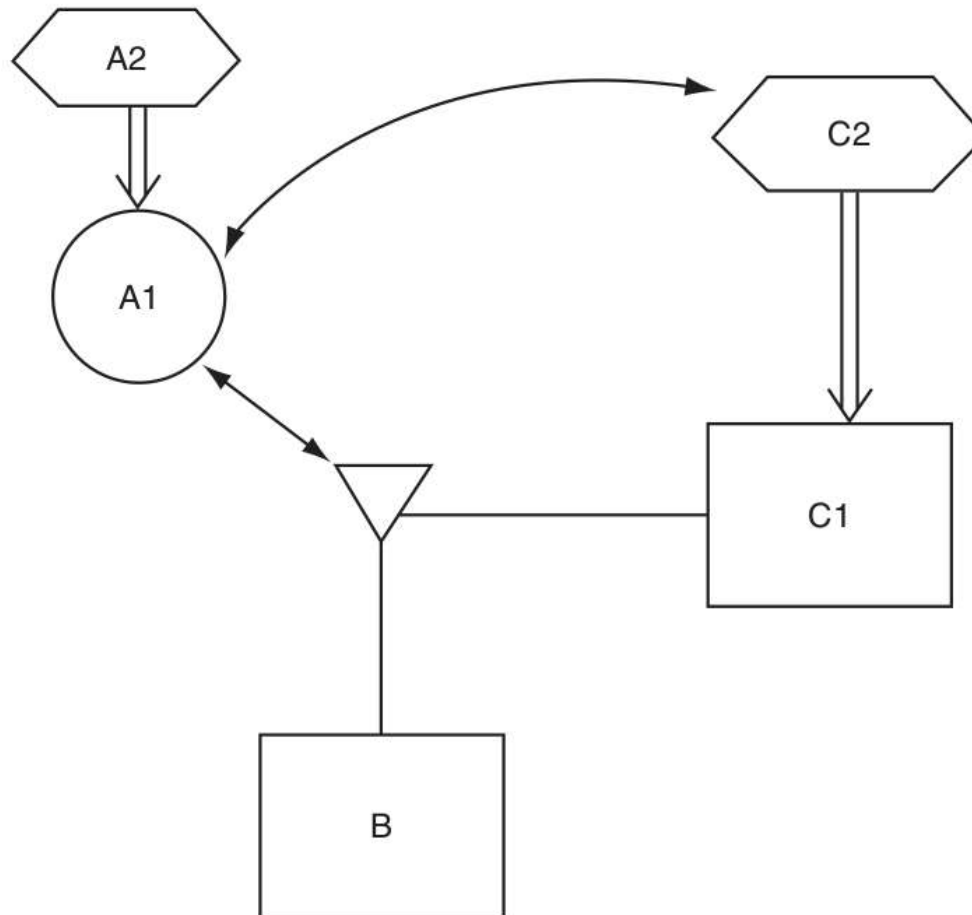
- Difficult to allocate to a hardware or software engineer
- A systems engineer
- Use for software validation

Hardware and Software Modeling - Single Model - Recommendation

- Usually used in pre-partitioned system exploration
- Reused in post-partitioned system models
- Use it as a very fast VP for software verification

Hardware and Software Modeling - Filter and Translate

- The A2 component's activity is filtered by A1 to generate its interaction with B and C
- A1 would communicate directly with other software components for performance reasons



Hardware and Software Modeling - Filter and Translate - Pros - Cons

- Pro - Clear separation of concerns
- Con - The software activity that affects the hardware needs to be identified and clearly specified
 - Explicit communication is not always there
 - Shared memory

Hardware and Software Modeling - Filter and Translate - Management

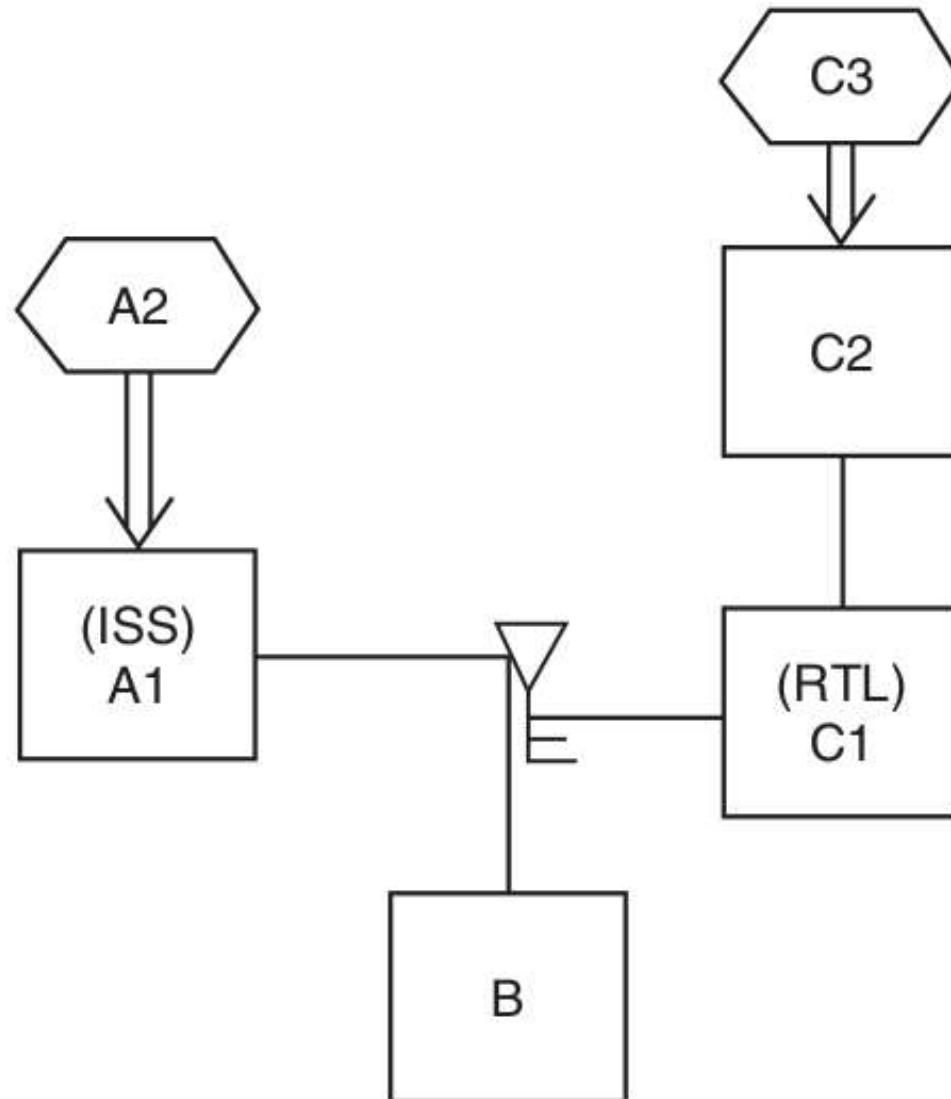
- A clear separation - responsibility is easy to assign
- Software engineers specify filter activity
- The hardware team usually models the filter component

Hardware and Software Modeling - Filter and Translate - Recommendation

- Most software is not aware of its effect on hardware
- Low-level software -> clear transactions
- The the software state mapping to hardware registers and bus activity is available to the interfacing component
- Useful when the software component is known early but its hardware platform is still malleable.

Hardware and Software Modeling - Separate Host Model

- Software components (A2 and C3) can be hosted by hardware component models (A1 and C2)



Hardware and Software Modeling - Separate Host Model -

Pros - Cons

- Pro - Used at many different abstraction levels
- Mostly low abstraction levels
- A very flexible style of co-modeling
- Con - Can be very slow

Hardware and Software Modeling - Separate Host Model - Management

- A clear separation of concerns
- The interface required between the hardware and software is needed for the implementation - the system design team

Hardware and Software Modeling - Separate Host Model - Recommendation

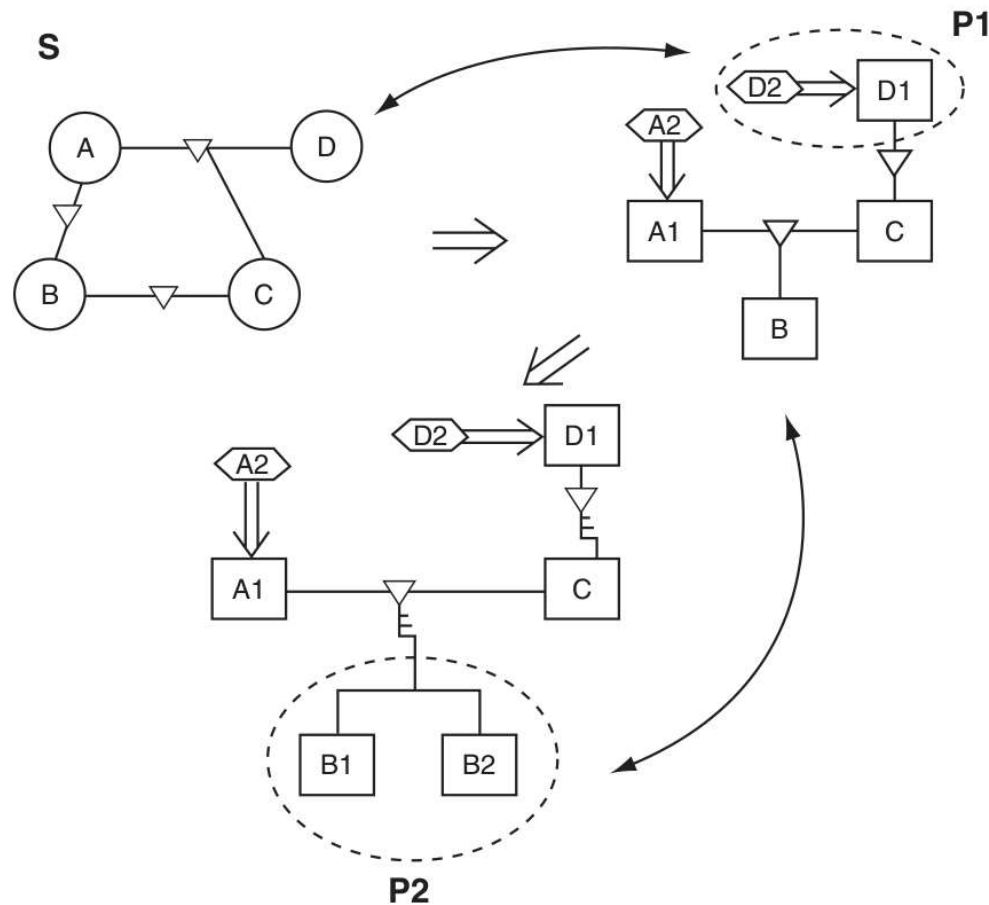
- The very-high-level models in this style - similar to the filtering
- A transition from style to style - can be made smoothly
 - Interfaces that translate between the different levels of abstraction

Pre-Partitioned Model Components I

- The modeling styles should be compatible
- A truly useful system model would be able to mix “pre” and “post” component models
- Clear separation
 - Internal functionality
 - External communication
- The communication can be refined independently or translated
 - Different abstraction level
 - Different computation model

Mixing Model Component Example

- With adaptive interfaces, D1 and D2 in P1 can be replaced by D - and vice versa - and, similarly, B in S, B in P1, and B1 and B2 in P2 can be substituted for each other.



Abstraction Levels I

- Untimed
- Loosely Timed
- Approximately Timed
- In addition to standard abstraction levels -> standard interfaces (APIs)
- Translating along the data axis is relatively straightforward
- Translating along the temporal axis is more difficult
- Information must be added when translating down...
- ... removed when translating up
- TLM 2.0

Dynamic and Static Analyses

- This is the first phase in which some realistic analysis can be done
- Wholly dynamic - the system models simulation
- Wholly static - the interface formal analysis and functional specifications
- Mostly hybrid analyses
 - Data static analysis
 - A submodel simulation produced from a static analysis

Dynamic and Static Analyses - Functional Analysis

- Validation and verification?
- Storage locations (queues, FIFOs) - become full
- Model other parts are delayed
- This analysis gathers data about queue sizes
- Data will be post-processed to see where the bottlenecks are occurring
- An analysis may lead to a re-partition

Dynamic and Static Analyses - Performance Analysis

- To validate the partitioning and hardware and software elements sizing.
- Realistic temporal performance annotations required
- Allocation of components to hardware and software -> concurrency and resource sharing idea
- The functionality and timing of many components may depends on the data
 - Real data - better performance estimates
- A dynamic data generation phase with post-process static analysis
- Some performance properties can be continually checked

Dynamic and Static Analyses - Interface Analyses

- Communication at an abstract level (not with wires or signals)
- For standard buses -> no problems
- For newly designed interfaces -> no guarantee for communication abstractions
- Check if new interfaces can even be implemented
- Mixed simulation with interface RTL model
- The new protocol should be designed and verified down to the lowest level

Dynamic and Static Analyses - Power Analyses

- the first which the model resembles a physical implementation
- Data activity and movement
- Graph - power consumed is allocated to components depending on their activity
- Although the power consumption estimate would **not be very accurate**, it would **give** some gross **idea** of where **problems could lie** in the implementation.
- Interface specifications at wire level gives a good power measure for communication
- A lot of data from simulation
- A lot of experience from the person performing the analysis
- Edge cases very important -> Peak data

Dynamic and Static Analyses - Area Analyses

- The hardware components complexity -> the final implementation probable area
- No previous data (experience) -> large margin of error
- The best - standard complexity metrics for function and interfaces
 - Previous experience and data
- New complexity metric
 - A previous project post-analysis
 - Scaling factors
- The most successful companies doing systems design - continuity between project
- Perform a postmortem

Dynamic and Static Analyses - Cost and Debug Capability Analyses

- Cost analysis uses heuristics to estimate complete lifetime costs
- Again depend on experience

Dynamic and Static Analyses - Debug Capabilities Analyses

- When a complex system is put onto a chip, the device external pins only observed point
- Over one half of all chips have functional errors
- Enable a bug or unexpected behavior in the chip to be analyzed and debugged
- Most of these capabilities require some additional logic
- DFT - Design For Test
- JTAG
- Normally inserted in the back end of the implementation process
- moving DFT up earlier in the design flow